

Mechanism-Aware AI Assurance: An Operational-Premise Taxonomy for Artificial Intelligence

Wesley R. Elsberry

September 26, 2026
Preprint

Abstract

AI assurance needs an explicit account of the computation that produces system behavior. We propose the Operational-Premise Taxonomy (OPT), a mechanism vocabulary with seven roots: **Lrn** for parametric learning, **Evo** for evolutionary adaptation, **Sym** for symbolic reasoning, **Prb** for probabilistic inference, **Sch** for search, planning, and solution-state optimization, **Ctl** for control and feedback, and **Swm** for swarm or collective behavior. A composition grammar describes hybrids. OPT-Intent records planned mechanisms, constraints, assumptions, and evidence needs; OPT-Code records the corresponding implementation description. Comparing these records is intended to make mechanism changes reviewable. This is a conceptual design paper supported by selected literature and retrospective failure illustrations. Existing governance frameworks already address methods, lifecycle risk, and hybrid systems; OPT offers a compact annotation that may help operationalize those obligations. No controlled study establishes that OPT improves classification accuracy, review efficiency, or failure prevention. The paper distinguishes proposed benefits from established evidence, specifies evidence and human-review boundaries, and outlines how those benefits could be tested.

1 Introduction

Current AI governance practice rightly emphasizes accountability, intended use, risk management, documentation, monitoring, and human oversight. Yet those processes can be incomplete if they do not identify the operative mechanism that creates system behavior. A risk register that labels a system only as “AI,” “machine learning,” or “automated decision support” may say little about the failure modes that matter for review. A learned model, a symbolic rule engine, a probabilistic inference system, a search optimizer, a feedback controller, an evolutionary search process, and a swarm system require different assurance evidence.

Supervised, unsupervised, and reinforcement learning describe aspects of learning problems; they do not exhaust the mechanisms studied in artificial intelligence [Russell and Norvig, 2020, Sutton and Barto, 2018]. This is not a claim that current governance instruments define AI only through those labels. The EU AI Act’s recital 12 explicitly includes logic- and knowledge-based approaches, and NIST AI RMF MAP 2.1 asks organizations to document implementation methods [European Union, 2024, Tabassi, 2023]. Learning-signal labels alone remain insufficient for mechanism-level assurance. They do not describe whether a deployed system reasons over

rules, updates a belief state, searches an objective landscape, controls an actuator, evolves a population, or coordinates through local collective behavior.

We propose an alternative taxonomic axis: the *operational premise*—the primary computational mechanism a system instantiates to improve, adapt, infer, optimize, decide, or act. The resulting *Operational-Premise Taxonomy* (OPT) provides a transparent and consistent framework for compactly describing AI systems, including hybrids and pipelines. OPT retains biological analogs (learning vs. adaptation) while accommodating symbolic, probabilistic, search, control, and swarm paradigms. Its purpose is not merely to name systems more neatly. Its purpose is to make mechanism-specific assurance questions visible at design time, implementation time, and review time.

The contributions of this paper are:

1. a seven-root mechanism taxonomy spanning **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, and **Swm**;
2. a compact hybrid grammar for systems that combine mechanisms;
3. mathematical and biological grounding for each root;
4. an assurance-oriented account of mechanism-specific and joint mechanism risk;
5. OPT–Intent and OPT–Code artifacts for design-time declaration, implementation-time classification, and drift review.

2 Method, Scope, and Evidence Status

This paper develops a design proposal through a targeted reading of AI textbooks, selected mathematical foundations, governance instruments, and software-failure reports. It is not a systematic literature review or an empirical validation of a taxonomy. The comparison asks what each inspected source actually describes, rather than treating the absence of OPT’s exact vocabulary as the absence of mechanism awareness.

The unit of classification is a named component at a specified lifecycle stage and version. A pretrained model can have a **Lrn** training history without changing its parameters during inference. A prompt that asks a model to “reason symbolically” does not establish a separate **Sym** engine; executable rules or another inspected symbolic procedure are needed to support that classification. Likewise, repeated agent calls do not by themselves establish the dynamics or stability properties of **Ctl**. Human approvals and ordinary transport, storage, hashing, and formatting steps belong in the workflow record even when an OPT root adds little to their description.

Evidence has three distinct roles here: established sources support descriptions of existing methods; historical reports support bounded failure descriptions; and author-designed examples illustrate proposed uses. None of these alone demonstrates OPT’s effectiveness. Appendix I records the basis and limits of the principal claims and figures. The evaluation agenda specifies how the proposed benefits could be tested.

3 The Assurance Gap: Making Mechanism Evidence Explicit

AI governance frameworks have become the dominant way to organize accountability for artificial intelligence systems. Frameworks such as the NIST AI Risk Management Framework, ISO/IEC

AI standards, OECD classification work, and the EU AI Act establish essential process controls: ownership, intended use, risk management, documentation, monitoring, human oversight, and post-deployment review. These controls are necessary. They are not sufficient when they remain mechanism-agnostic.

The missing layer is mechanism-level assurance. A system can be documented, assigned an owner, placed in a risk tier, and subjected to lifecycle review while still failing to ask whether the operative computational mechanism has known failure modes that matter in the deployment context. Mechanism is not a minor implementation detail. A learned ranking model, a probabilistic nowcasting system, a search optimizer, a rule engine, and a feedback controller create different assurance questions even when each is labeled “AI” or “automated decision support.”

Selected software-intensive failures illustrate the importance of checking operational assumptions. Ariane 5 Flight 501 exposed numeric-range assumptions in reused flight software [[Ariane 501 Inquiry Board, 1996](#)]. Mars Climate Orbiter exposed a mismatch between the units supplied by ground software and those required at an interface [[Mars Climate Orbiter Mishap Investigation Board, 1999](#)]. The SEC’s Knight Capital order describes inconsistent deployment and activation of obsolete code [[U.S. Securities and Exchange Commission, 2013](#)]. The NTSB’s 2019 MCAS safety-recommendation report examines sensor inputs and assumptions about pilot responses to unintended system activation; it is not a complete accident-causation report [[National Transportation Safety Board, 2019](#)]. The 2003 Northeast blackout report identifies alarm and situational-awareness failures [[U.S.-Canada Power System Outage Task Force, 2004](#)]. The 2010 flash-crash report describes volume-responsive automated selling interacting with market activity and liquidity [[U.S. Commodity Futures Trading Commission and U.S. Securities and Exchange Commission, 2010](#)]. The Google Flu Trends analysis documents prediction errors and discusses changing search behavior, platform changes, and recalibration [[Lazer et al., 2014](#)].

These selected cases do not establish the frequency of a common failure mechanism, an OPT classification of each entire system, or that OPT would have prevented the incidents. They motivate questions about operational assumptions. The proposal is that mechanism-aware review can organize such questions; whether it improves detection or makes assumptions visible earlier requires evaluation. Once a system is classified as **Lrn**, **Prb**, **Sch**, **Ctl**, **Sym**, **Evo**, **Swm**, or a hybrid, assurance can ask mechanism-specific questions:

- For **Lrn**, what evidence addresses biased labels, leakage, distribution shift, reward or loss misspecification, and calibration?
- For **Prb**, are priors, likelihood assumptions, uncertainty estimates, and approximations valid in the operational setting?
- For **Sch**, what objective is optimized, which constraints are hard, and what prevents pathological optima?
- For **Ctl**, what are the stability margins, sensor assumptions, actuator authority, degraded modes, and human override conditions?
- For **Sym**, are rules traceable, consistent, complete enough for use, and robust to exceptions?
- For **Evo**, does the fitness function represent the desired outcome, and what prevents unsafe candidate exploration?
- For **Swm**, what local rules generate the global behavior, and what bounds emergent instability?

OPT therefore addresses an assurance gap rather than replacing governance. Governance frameworks specify who is accountable and what kinds of evidence must exist. OPT helps identify which evidence is relevant by naming the operative mechanism and the joint risk profile of hybrids. OPT-Intent extends that benefit to design time by recording intended mechanisms, assumptions, constraints, risks, and release-blocking evidence before implementation hardens around implicit choices. OPT-Code records the mechanisms actually present in the implemented system. Comparing the two exposes mechanism drift: for example, a nominally symbolic system that quietly gains learned ranking, a decision-support system that becomes de facto control, or a forecasting model that becomes part of an optimization loop.

This framing also explains why training-signal labels are inadequate. The problem is not only that “supervised,” “unsupervised,” and “reinforcement” omit many AI traditions. The deeper problem is that such labels are too coarse to drive assurance. A governance process needs a compact way to ask what kind of computation produces behavior, how mechanisms combine, what failure modes follow from that composition, and which tests or monitors make residual risk explicit. OPT is proposed as that mechanism layer.

4 Operational-Premise Taxonomy (OPT)

Because OPT introduces several new labels, we present those here before tackling background and related work topics. (See also Appendix A.)

OPT classes are defined by dominant mechanism; hybrids are explicit compositions:

- **Learnon (Lrn)** — Parametric learning within an individual (gradient/likelihood/return updates).
- **Evolon (Evo)** — Population adaptation via variation, selection, inheritance.
- **Symbion (Sym)** — Symbolic/logic inference over discrete structures (KB, clauses, proofs).
- **Probion (Prb)** — Probabilistic modeling and approximate inference (posteriors, ELBO).
- **Scholon (Sch)** — Search, planning, and solution-state optimization, including continuous constraint relaxation.
- **Controlon (Ctl)** — Feedback control and state estimation in dynamical systems.
- **Swarmon (Swm)** — Collective/swarm coordination with local rules and emergence.

Scope of Sch. Continuous relaxation qualifies when evolving states represent candidate solutions to an encoded problem. Analytically configuring its coefficients does not establish **Lrn**; an implemented parameter learning phase must be identified separately, whether offline or online. An optimization objective alone does not add **Sch** to **Lrn**, **Evo**, or **Swm**; composition requires evidence of distinct operative mechanisms.

Hybrid notation. We use $A+B$ for co-operative mechanisms, A/B for hierarchical nesting (outer/inner), $A\{B,C\}$ for parallel ensembles, and $[A\rightarrow B]$ for pipelines (Appendix B).

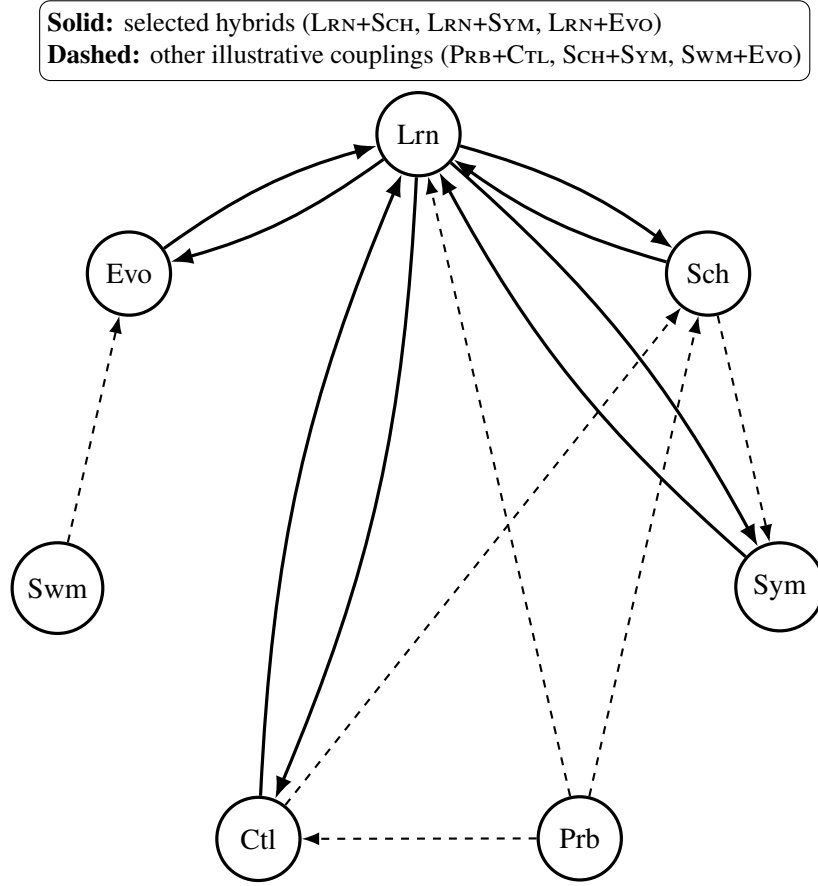


Figure 1: Author-designed schematic, not a frequency or performance plot. OPT roots: **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, **Swm**.

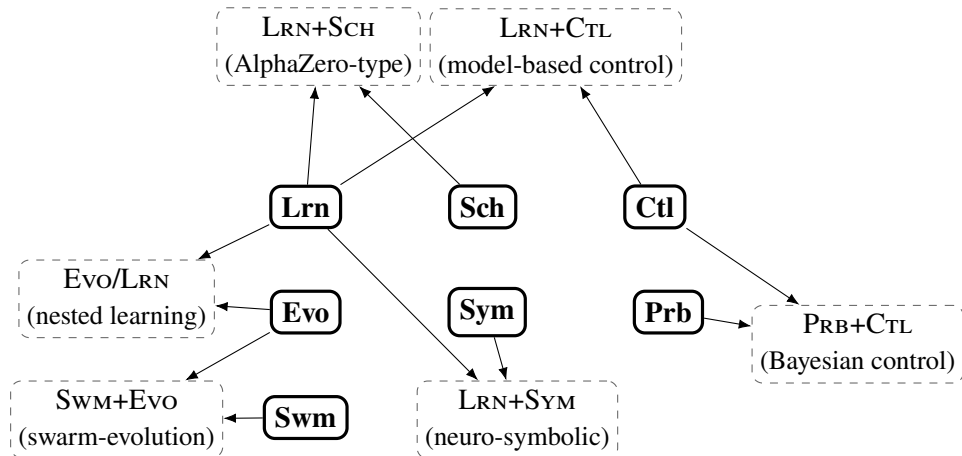


Figure 2: Author-designed composition schematic: roots (solid boxes) and illustrative hybrids (dashed boxes). Lines denote components, not historical ancestry or measured relationships.

5 Background and Prior Work

The literature spans several computational traditions. A general AI textbook covers search, logic, probability, and learning [Russell and Norvig, 2020]; a computational-intelligence text treats neural, fuzzy, and evolutionary methods [Keller et al., 2016]; a software survey addresses fuzzy systems [Alcala-Fdez and Alonso, 2016]; and a reinforcement-learning text distinguishes learning from planning [Sutton and Barto, 2018]. These sources establish breadth, not a shared endorsement of OPT’s seven categories. No-Free-Lunch (NFL) theorems for search/optimization motivate pluralism: averaging uniformly over the relevant objective-function space yields no universally superior optimizer under the theorem’s assumptions [Wolpert and Macready, 1997]. Biological mechanisms and computational models offer selected correspondences: synaptic plasticity and Hebbian/Oja learning [Hebb, 1949, Oja, 1982], population genetics and replicator dynamics [Price, 1970, Taylor and Jonker, 1978], Bayesian cognition [Knill and Pouget, 2004], and optimal feedback control in motor behavior [Todorov and Jordan, 2002, Kalman, 1960, Pontryagin et al., 1962].

6 Related Work: Existing Taxonomies and Frameworks

The sources below serve different purposes. They document methods, risks, system composition, and design decisions. OPT’s proposed contribution is a compact composition notation and a paired record for planned and implemented mechanisms within that existing practice.

Standards and terminology. ISO/IEC 22989:2022 provides AI concepts and terminology; ISO/IEC 23053:2022 describes a framework for systems using machine learning [ISO/IEC, 2022a,b]. The descriptions here are limited to the standards’ public scope statements; the complete standards were not inspected. No clause-level conclusion about hybrid handling or non-ML integration follows from those scope statements.

Risk and use frameworks. NIST AI RMF 1.0 explicitly includes methods, scientific integrity, component risks, and documented evaluation. Its MAP 2.1, MAP 2.3, and MAP 4 are natural places to attach OPT records [Tabassi, 2023]. OPT’s added value would need to be tested against a competent implementation of those existing requirements. NIST’s AI Use Taxonomy organizes human–AI activities, a different descriptive axis [Theofanos et al., 2024].

Policy classification and regulation. The OECD framework includes an AI-model dimension and explicitly distinguishes symbolic, statistical, and hybrid models; it also discusses interlinked models and composite systems [OECD, 2022, pp. 18, 44–46]. OPT offers finer mechanism labels and composition syntax for an annotation within such a profile. The EU AI Act addresses obligations for systems and models, including technical documentation; its definition is not limited to machine learning [European Union, 2024, recital 12 and Annex IV]. This paper does not offer a legal compliance determination.

Documentation and architectural decisions. Model Cards describe intended uses, evaluation conditions, and limitations of trained models [Mitchell et al., 2019]. Architecture decision records preserve context, decisions, status, and consequences, including decisions made during development [Nygard, 2011]. Both can contain mechanism information. OPT–Intent is a

proposed structured annotation for such records, not the first way to document design-time decisions.

Computational foundations. Search, logic, probability, learning, control, and evolutionary computation have substantial prior literatures [Russell and Norvig, 2020, Keller et al., 2016, Sutton and Barto, 2018]. The biological correspondences in Section 13 are illustrative and sometimes model-based. They do not establish exclusive natural categories. No-Free-Lunch results apply under specified averaging assumptions; they neither prove OPT’s seven roots necessary nor establish this taxonomy’s completeness [Wolpert and Macready, 1997].

Comparative landscape. Table 1 summarizes the inspected sources and the proposed relationship to OPT. It compares purposes and evidence requirements rather than awarding binary capability scores to frameworks with different scopes.

Table 1: Evidence-bounded comparison. Entries describe scope, not benchmark results.

Source	Existing contribution	Relationship to OPT
OPT (proposal)	Seven roots, hybrid syntax, Intent/Code records.	Proposed annotation and review aid; benefits not validated.
ISO/IEC 22989 and 23053 [ISO/IEC, 2022a,b]	AI terminology and ML-system framework.	Scope-page comparison only; no claim that the full texts exclude hybrids.
NIST AI RMF [Tabassi, 2023]	Methods, component risks, lifecycle evaluation.	OPT records could organize evidence already requested by MAP and MEASURE.
NIST AI Use Taxonomy [Theofanos et al., 2024]	Human–AI task and use descriptions.	Mechanism annotations address a different axis.
OECD classification [OECD, 2022]	Multi-axis profile including symbolic, statistical, and hybrid models.	OPT could add a compact description of particular compositions.
EU AI Act [European Union, 2024]	Obligations and technical documentation.	An OPT record is supplementary, not evidence of compliance by itself.
Model Cards; ADRs [Mitchell et al., 2019, Nygard, 2011]	Model evaluation and design-decision records.	Can host Intent/Code annotations; already support relevant documentation.
AI textbooks [Russell and Norvig, 2020]	Established computational families and algorithms.	Main conceptual antecedents; OPT proposes a shared notation.

7 Comparative Analysis, Completeness, and Objections

7.1 Biological–Artificial Correspondences

Each OPT class is associated here with a biological mechanism or computational model (plasticity, natural selection, structured reasoning, Bayesian cognition, deliberative planning, optimal feedback control, and distributed coordination). Shared operators in Sec. 13 motivate comparisons; any transfer of a guarantee requires its mathematical assumptions to hold in both settings.

7.2 Coverage, Hybrids, and Orthogonal Descriptors

Hybrids are explicit (e.g., L_{RN}+S_{CH} for the AlphaZero learning/search workflow [Silver et al., 2017], L_{RN}+S_{YM} neuro-symbolic, E_{VO}/L_{RN} evolutionary selection enclosing an individual-learning procedure). Pure evolutionary optimization of neural parameters need not contain a separate L_{RN} update. Orthogonal axes capture representation, locus of change, objective, data regime, timescale, and human participation.

7.3 Objections and Responses

Reduction to optimization. Mechanisms imply distinct guarantees/hazards (data leakage vs. fitness misspecification vs. rule brittleness). NFL does not establish the number or boundaries of OPT roots. **Hybrid blurring.** OPT treats compositions as first-class; the notation discloses “what changes where, on what objective, and on what timescale.” **Regulatory simplicity.** Seven roots are a design choice. Minimality, completeness, and the practical readability of the notation remain evaluation questions.

8 Examples and Mapping

A learned model’s training history must be distinguished from inference at fixed parameters. Planning, probabilistic inference, and control are not automatic additions to every reinforcement-learning system. For Kalman filtering the broad estimation sense of **Ctl** overlaps **Prb**; this is a boundary to document, not evidence of two separately implemented components.

9 Orthogonal Axes and Risk Perspectives

Secondary axes (orthogonal descriptors).

- **Representation:** parametric vectors, symbols/logic, graphs, programs, trajectories, policies.
- **Locus of Change:** parameters, structure/architecture, population composition, belief state, policy.
- **Objective Type:** prediction, optimization, inference, control, search cost, constraint satisfaction.
- **Timescale:** online vs. offline; within-run vs. across-generations.
- **Data Regime:** none/synthetic, labeled, unlabeled, interactive reward.

Table 2: Illustrative mappings by operation; additions require implementation evidence.

Type / Implementation	Examples	OPT (short)
Neural-network training	Gradient-based fitting	Lrn
Learning values/policies	val- DQN, policy gradient	Lrn
Evolutionary algorithms	algo- GA, GP, CMA-ES	Evo
Collective optimization	ACO, PSO	Swm
Expert systems	Prolog, Mycin, XCON	Sym
Probabilistic models	BN, HMM, factor graphs	Prb
Search & solution optimization	A*, MCTS, STRIPS, Hopfield–Tank solving	Sch
Feedback control	PID, LQR, MPC	Ctl
Probabilistic estimation	Kalman filtering	Prb; Ctl overlap

- **Human Participation:** expert-authored knowledge vs. learned vs. co-created.

9.1 Artificial Immune Systems (AIS) in OPT

Artificial immune systems are a useful boundary test for a mechanism taxonomy: shared biological inspiration does not establish a shared implementation. The following classifications are this paper’s interpretations of specified algorithmic operations, not labels supplied by the source authors.

Clonal selection. CLONALG selects high-affinity candidates, clones them, mutates clones, and reselects survivors [de Castro and Zuben, 2002, Sec. VI]. These operations support **Evo**. They do not by themselves establish **Swm** interaction or **Prb** inference. A normalized Hamming affinity $1 - d_H(x, a)/L$ is one possible score for length- L bitstrings, not a calibrated probability; the actual affinity and mutation rule must be recorded. An immune-network variant needs its own interaction evidence before receiving a collective label.

Negative selection. The original detector procedure rejects candidate strings that match protected “self” strings and later tests retained detectors against observed strings [Forrest et al., 1994, Sec. 2]. Detector generation and runtime matching are different phases. A probabilistic analysis of detection rates does not automatically turn the matcher into a probabilistic-inference engine. Neither random candidate generation nor the word “selection” establishes evolutionary reproduction.

For a matching relation $m(d, x)$, a meaningful no-self-match condition is $\forall d \in D, \forall s \in S :$

Table 3: Orthogonal descriptive axes and governance risks (abridged).

OPT	Primary Risks	Assurance Focus
Lrn	Data leakage, reward hacking	Data governance, OOD tests, calibration
Evo	Fitness misspecification	Proxy validation, replicates, constraints
Sym	Rule brittleness, KB inconsistency	Provenance, formal verification
Prb	Miscalibration, inference bias	Posterior predictive checks
Sch	Heuristic bias, local minima, invalid solutions	Objective checks, feasibility tests, optimality analysis
Ctl	Instability, unmodeled dynamics	Stability margins, robustness
Swm	Emergent instability	Swarm invariants, safety envelopes

$\neg m(d, s)$, not simply $d \notin \mathcal{S}$. A proposed coverage target is

$$\Pr_{x \sim Q} [\exists d \in D : m(d, x) \mid x \notin \mathcal{S}] \geq \tau,$$

where Q is a declared evaluation distribution, $\Pr_Q(x \notin \mathcal{S}) > 0$, and τ a chosen target. This is an assurance criterion proposed here, not an achieved result or a universal guarantee of negative selection.

Dendritic-cell processing. The deterministic DCA transforms common safe/danger signals, accumulates cell contexts over lifespans, and aggregates antigen-associated scores [Greensmith and Aickelin, 2008, Sec. 3, Algorithm 1]. Weighted signals and normalized scores alone do not establish posterior inference, and a cell population alone does not establish local agent interaction. These operations do not provide a sufficient basis for an unconditional **Swm+Prb** assignment under the definitions used here. Classification remains unresolved pending explicit inclusion criteria; the operational description remains available for assurance review.

Immune networks and assurance. A specific network with demonstrable local stimulation/suppression may motivate **Swm**; add **Evo** only for implemented variation and selection, and **Ctl** only for the specified regulation process. An invented differential-equation sketch is not evidence of a published immune-network implementation and is omitted here. Review detector coverage and false alarms on held-out inputs, sensitivity to thresholds and data drift, and interaction stability where applicable. Calibrate any claimed probabilities separately. These checks address mechanisms and assumptions; no OPT intervention result is reported.

10 Design and Governance with OPT–Intent and OPT–Code

A central motivation for the Operational Premise Taxonomy (OPT) is to support not only the analysis of existing AI systems, but also the design and governance of systems throughout

their lifecycle. Existing documentation and architecture practices can span the lifecycle. OPT proposes a focused mechanism annotation within those practices, with a record of what evidence supports each assignment.

To this end, we distinguish two complementary artefacts: *OPT-Intent*, a design-time declaration of planned mechanisms, goals, constraints, and risks; and *OPT-Code*, a run-time classification of the system as implemented. Together, these artefacts form a governance substrate that is lightweight, expressive, and compatible with software-architecture practices and AI governance frameworks. Their assurance role is to tie general governance obligations to mechanism-specific evidence: the tests, monitors, constraints, and residual-risk decisions that are relevant because of how the system actually computes.

10.1 OPT-Intent as a Design-Time Mechanism Declaration

OPT-Intent expresses the *intended* operative mechanisms (**Lrn, Evo, Sym, Prb, Sch, Ctl, Swm**), the domain-level goal, key constraints, anticipated risks, and the deployment context. The notation supports early-stage architectural reasoning:

```

INTENT-OPT = Sch/Evo
INTENT-GOAL = robust-production-schedule-under-dynamic-constraints
INTENT-CONSTRAINTS = real-time, explainable, limited-human-oversight
INTENT-RISKS = local-minima, premature-convergence,
objective-misspecification
INTENT-EVIDENCE = constraint-tests, stress-cases, monitor-thresholds
INTENT-CONTEXT = manufacturing-decision-support

```

This declaration resembles a focused architectural decision record (ADR), but is grounded in OPT’s mechanism vocabulary. Whereas classical goal-oriented requirements engineering (GORE) frameworks such as KAOS, i, or Tropos provide high-level goal models, OPT-Intent provides a mechanism-centered annotation that connects those goals to families of computational approaches.

10.2 OPT-Code as an Implementation-Time Mechanism Description

Once a system is implemented, its operative mechanisms can be classified via OPT-Code:

```

OPT=Evo/Sch/Sym; Rep=permutations+rules; Obj=production-cost;
Data=inventory+constraints;
Time=generations+online-adjust; Human=medium

```

OPT-Code reflects the *actual* mechanisms as they appear in the final architecture and implementation. Comparing OPT-Intent with OPT-Code provides a principled way to detect architectural drift, unplanned mechanism additions, and deviations from original constraints.

10.3 Alignment Analysis: Intent vs. Implementation

The relationship between OPT-Intent and OPT-Code supports alignment analysis across the AI system lifecycle:

- **Mechanism alignment:** Whether the realized mechanisms match the intended roots, or whether additions (e.g., **Sym** for explainability) or substitutions introduce new behavior.

- **Objective alignment:** Whether the objective in OPT–Code (Obj) is consistent with the purpose in INTENT–GOAL.
- **Constraint alignment:** Whether the implementation respects INTENT–CONSTRAINTS (e.g., real-time, explainability, human oversight).
- **Risk evolution:** Whether realized mechanisms introduce additional risks relative to INTENT–RISKS (e.g., adding learned components introduces data-dependence).

A proposed LLM-based evaluator can generate an alignment verdict and rubric score for review. This is a candidate judgment, not verified drift detection; code and execution evidence remain necessary.

10.4 Integration with Existing Governance Artefacts

OPT aligns with and supplements existing governance frameworks:

AI Documentation (Model Cards, AI Service Cards). Model Cards and similar frameworks capture purpose, data provenance, limitations, and performance characteristics of trained models. OPT–Intent can supplement them with a *design origin*, and OPT–Code provides a *mechanism-centric summary* useful for governance, reproducibility, and safety assessments.

Architecture Decision Records (ADRs). ADRs record the rationale for major architectural decisions. OPT–Intent functions as a structured, mechanism-focused ADR, intended to be referenced in downstream ADRs describing implementation choices and trade-offs.

Safety, Risk, and Impact Assessments. Regulatory frameworks such as the OECD AI classification or NIST AI Risk Management Framework classify AI systems according to use, risk, and context. OPT complements these by classifying operative mechanisms. Mechanism-level classification is critical because risk profiles are often mechanism-dependent: population-based adaptation (**Evo**), closed-loop control (**Ctl**), probabilistic inference (**Prb**), search/optimization (**Sch**), and learned representation (**Lrn**) each generate distinct failure modes and therefore require distinct assurance evidence.

GORE and Requirements Engineering. OPT–Intent is compatible with KAOS, i, and Tropos goal structures, providing a compact mapping from stakeholder goals to operative mechanisms. Instead of treating “use AI” as a monolithic design choice, OPT forces the mechanism to be named explicitly.

10.5 Lifecycle Governance with OPT

An AI system moves through phases of design, implementation, deployment, revision, and decommissioning. OPT supports governance at each phase:

1. **Design:** Authors specify OPT–Intent and identify mechanistic justifications and constraints.
2. **Implementation:** OPT–Code is generated and compared with Intent for architectural drift.

3. **Evaluation:** OPT classifiers, evaluators, and adjudicators check mechanism correctness, formatting, and risk implications.
4. **Deployment:** OPT–Code informs safety monitoring, audit logs, and mechanism-specific risk controls (e.g., for **Ctl** or **Evo** systems).
5. **Revision and re-training:** OPT alignment is reassessed when system behavior changes or new mechanisms are introduced.
6. **Documentation & reporting:** OPT–Intent and OPT–Code form part of a long-term audit trail, linking design rationale to implemented system behavior.

10.6 AI Design Assistants and Automated Governance

OPT also provides a structured interface for LLM-based design assistants. Given a functional goal or stakeholder requirement, an OPT-aware model can produce candidate OPT–Intent declarations and propose mechanism families suitable for achieving the goal. Downstream evaluation and adjudication prompts make it possible to manage and audit these proposals automatically.

Such workflows are intended to support mechanism-level traceability. Instead of asking only whether a system is “fair,” “safe,” or “performant,” practitioners can ask whether its mechanisms match the intended design, whether mechanism additions add new risks, and whether the alignment between purpose and implementation is conserved over time. OPT thus becomes a bridge between requirements engineering, architectural practice, risk governance, and the technical analysis of AI systems.

11 OPT and Agentic AI Workflows

Status. This section describes proposed uses, not measured capabilities of a validated OPT agent. Self-reported classifications and model scores require independent checks.

Recent advances in agentic artificial intelligence emphasize systems that plan, act, evaluate, and repair their own behavior through iterative interaction with tools, environments, and internal models. Such systems typically decompose goals, invoke tools, assess outcomes, and revise plans in closed loops. An architecture description may omit an explicit representation of the *operative mechanisms* through which actions are taken and errors arise. This omission complicates reasoning about failure modes, governance constraints, and design trade-offs.

The Operational Premise Taxonomy (OPT) provides a mechanism-level abstraction layer that can be integrated into agentic workflows to address these gaps. Rather than prescribing a particular agent architecture, OPT supplies a shared vocabulary and analytical framework that agentic systems can use to reason about how tasks are performed, how errors should be interpreted, and how repairs should be constrained.

11.1 Mechanism Awareness in Agentic Systems

Agentic workflows are often described in terms of high-level functional stages (planning, execution, critique, repair), but these stages are agnostic to the computational mechanisms employed. In practice, however, the behavior and risk profile of an agentic system depend critically on whether its actions rely on parametric learning (**Lrn**), symbolic reasoning (**Sym**),

search (**Sch**), probabilistic inference (**Prb**), control (**Ctl**), evolutionary adaptation (**Evo**), or swarm dynamics (**Swm**), or some hybrid combination thereof.

OPT introduces explicit mechanism awareness into agentic reasoning. An OPT-aware agent can classify its own components, tools, or subplans in terms of OPT roots, enabling it to reason not merely about *what* is being done, but about *how* it is being done. This distinction becomes especially important in hybrid agentic systems that combine learning-based components with search, symbolic constraints, or control loops.

11.2 OPT-Intent in Agentic Planning

During goal intake and planning, agentic systems must decide not only which actions to take, but which classes of computational strategies are appropriate. OPT-Intent provides a compact way to express these design-time commitments. An OPT-Intent declaration specifies the intended operative mechanisms, the system’s goal, relevant constraints, and anticipated risks.

In an agentic context, OPT-Intent functions as a planning constraint. It guides the selection of tools and strategies, discourages unprincipled mechanism substitution (e.g., defaulting to learning-based solutions when symbolic or search-based approaches are more appropriate), and provides an explicit reference against which subsequent behavior can be evaluated.

11.3 OPT-Code and Runtime Self-Description

As an agent executes plans and invokes tools, its effective operative mechanisms may diverge from those originally intended. OPT-Code provides a runtime or post-hoc description of the mechanisms actually employed. In agentic systems, this enables self-description and introspection: the agent can record and report which mechanisms were used to achieve a result.

Comparing OPT-Code against OPT-Intent enables the detection of *mechanism drift*, where new mechanisms are introduced implicitly or intended mechanisms are bypassed. This capability is particularly relevant for long-running or self-modifying agentic systems, where accumulated changes can undermine assumptions about safety, explainability, or compliance.

11.4 Mechanism-Guided Error Interpretation

A central challenge in agentic AI is automated error remediation. Errors in agentic systems are often diagnosed at the surface level (e.g., “the output was incorrect”), without regard to the underlying mechanism that produced the error. OPT enables mechanism-guided error interpretation by associating distinct classes of failure modes with different operative premises.

For example, failures in **Lrn**-dominated systems often involve generalization error or distributional shift, while failures in **Sch** systems may involve heuristic bias or combinatorial explosion. Control-oriented systems (**Ctl**) are prone to instability or oscillation, and evolutionary systems (**Evo**) may suffer from premature convergence or loss of diversity. By classifying the operative mechanism, an agent can narrow the space of plausible diagnoses and select repair strategies that are appropriate to the mechanism in use.

11.5 Constraint-Preserving Repair and Governance

OPT also supports constraint-aware repair. In governance-sensitive contexts, repairs must not introduce new operative mechanisms without justification, as doing so may alter the system’s

risk profile or regulatory status. An OPT-aware agent can evaluate proposed repairs against OPT-Intent to determine whether they preserve or violate intended constraints.

This capability enables a form of *mechanism-level governance* within agentic workflows. Rather than relying solely on external oversight, agents can self-monitor compliance with declared mechanism constraints, flag deviations, and require explicit authorization for changes that introduce new operative premises.

11.6 Multi-Agent Differentiation and Coordination

In multi-agent systems, OPT provides a principled basis for role differentiation and coordination. Agents may be specialized according to dominant operative mechanisms (e.g., search-focused agents, symbolic-reasoning agents, or learning-focused agents), with the intended, untested benefits of reducing reviewer effort and improving interpretability. OPT also provides a shared vocabulary for resolving conflicts when agents propose incompatible strategies, enabling negotiation in terms of mechanism trade-offs rather than ad hoc preferences.

11.7 Implications for Agentic AI Design

Incorporating OPT into agentic workflows does not require abandoning existing architectures. Instead, OPT functions as an intermediate abstraction layer that connects goals, mechanisms, and outcomes. By making operative premises explicit, OPT is intended to support planning, diagnosis, and governance. Whether it improves these outcomes is an empirical question.

As agentic AI continues to move toward greater autonomy and complexity, the ability to reason explicitly about operative mechanisms will become increasingly important. OPT offers a structured and extensible framework for supporting this capability within both single-agent and multi-agent systems.

12 Discussion: Mechanism-Aware Assurance Beyond Signal-Based Taxonomies

Mechanism clarity. Lrn-Swm encode distinct improvement/decision operators (gradient, selection, resolution, inference, search, feedback, collective rules).

Biological alignment. The biological correspondences include mechanisms, hypotheses and computational models of different evidential status. They motivate comparisons rather than establishing a common neural implementation.

Coverage hypothesis. The seven roots organize the examples examined here. A broader annotated corpus is needed to test coverage, ambiguity, and whether additional categories are useful.

Governance usability. Mechanism-aware controls attach naturally per class (Table 3). This is the central assurance value: OPT helps translate broad governance obligations into mechanism-specific questions about evidence, tests, monitoring, and residual-risk ownership.

12.1 Reclassification of Classic Systems

Table 4 records illustrative placements; a definitive assignment would require the implementation evidence and lifecycle scope specified in Section 2.

Table 4: Illustrative placements of classic systems. Classification depends on the component and lifecycle stage; names alone do not establish an implementation.

System	Prior label	OPT (short)
R1 (1980)	Expert system	Sym
CLIPS rule engine	Expert shell	Sym
Instar/Outstar learning	Neural rules	Lrn
Backprop	Supervised NN	Lrn
ART 1/2 weight learning	Unsupervised NN	Lrn
LMS/ADALINE	Supervised NN	Lrn
Hopfield–Tank TSP	Recurrent neural network	Sch (solving phase)
Boltzmann-machine training / sampling	Energy-based model	Lrn training; Prb sampling
Fuzzy Logic Control	Soft computing	Ctl (+ Sym)
Genetic Algorithms	Evolutionary	Evo
Genetic Programming	Program induction	Evo
Genetic-programming sym-bolic regression	Model discovery	Evo ; Sym conditional
PSO	Swarm optimization	Swm
A* search	Path finding	Sch
STRIPS, Graphplan	Search/planning	Sch (+ Sym)
Kalman filtering	State estimation	Prb ; Ctl overlap
LQR	Feedback control	Ctl

Source scope for the classic examples. R1’s production rules and the CLIPS rule engine motivate the **Sym** placements [McDermott, 1980, Secret Society Software, LLC, 2025]. Instar/outstar learning, backpropagation, ART weight adaptation, and ADALINE adaptation motivate the **Lrn** rows [Grossberg, 1986, Rumelhart et al., 1986, Carpenter and Grossberg, 1987a,b, Widrow and Hoff, 1960]. The ART papers also describe category search and reset; the learning row does not classify every operation of the complete architecture. A*, STRIPS, and Graphplan supply search/planning examples [Hart et al., 1968, Fikes and Nilsson, 1971, Blum and Furst, 1997]. These placements make no claim about universal search efficiency. The labels are this paper’s interpretations of the cited mechanisms, not classifications supplied by those sources.

Boundary cases. The canonical Hopfield–Tank TSP network is an artificial neural network whose computation uses recurrent neural dynamics for constraint satisfaction and optimization [Hopfield and Tank, 1985, pp. 144, 146–147]. Connections and biases are configured before solving to encode tour constraints and distances. During the canonical solving phase, neuron states change while those weights remain fixed; this configuration is not itself parameter learning

from examples. Thus neural computation is present without online weight adaptation. The task has the form of search, implemented by continuous state relaxation. Under the stated inclusion rule, OPT assigns **Sch** to this solving phase: neuron states encode candidate solutions and evolve under the configured objective. This assignment describes a neural computation without presuming a learning phase. A low-energy state does not guarantee a feasible or globally shortest tour. Gain schedules discussed in the paper are also distinct from weight learning [Hopfield and Tank, 1985, pp. 149–150].

We propose using failure modes as a cross-check on mechanism assignments. For this network, distinguish inadequate constraint penalties from failure to break initial-state symmetry and entrapment in local minima [Hopfield and Tank, 1985, pp. 146–150]. These suggest checks of the encoded objective, initialization, and solution validity. They do not establish that failures are concentrated in weight configuration or that this configuration is learning. Local-minimum failures occur in both parameter fitting and solution search; the changing variable and update procedure distinguish those operations. An extension that fits reusable weights from experience would require a separately described learning phase.

Boltzmann-machine weight learning and stochastic state sampling require different descriptions [Ackley et al., 1985]. PSO does not receive **Evo** solely because it maintains a population [Kennedy and Eberhart, 1995]. In genetic programming, a symbolic expression is a representation; add **Sym** only for an implemented symbolic operation, not from the task name “symbolic regression.” The other classic rows remain illustrative family-level placements, not source-code audits or validated reference labels.

12.2 Evaluation needed before effectiveness claims

A prospective study should compare the same assurance tasks under ordinary architecture and governance documentation and under that documentation supplemented with OPT–Intent and OPT–Code. A further arm could examine enforced conformance checks, separating the contribution of the notation from that of enforcement. Cases should include known mechanism changes, missing assumptions, component-interaction defects, and conforming implementations. Reviewers should work from a common evidence package, with a separately adjudicated reference and counterbalanced task order.

Report classification agreement, missed defects, false alarms, correction quality, elapsed review effort, and uncertainty. Record component versions, reviewer expertise, any model assistance, adjudication rules, and exclusions. No such study is reported here. A useful negative result would be that OPT adds effort without improving defect detection; that result should be retained rather than interpreted away.

13 Mathematical Foundations and Biological Correspondences

These equations illustrate selected mechanisms, not a derivation of the seven-root partition. Their assumptions and algorithm variants matter; OPT labels do not transfer theorems between systems.

Learnon (Lrn). For a sample $S = \{(x_i, y_i)\}_{i=1}^n$, a regularized empirical-risk objective is

$$\hat{\theta} \in \arg \min_{\theta} \left\{ \widehat{R}_S(\theta) + \lambda \Omega(\theta) \right\}, \quad \widehat{R}_S(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i). \quad (1)$$

Here f_{θ} is the predictor, ℓ the loss, Ω a penalty, and $\lambda \geq 0$ its weight. Population risk instead uses $R_{\mathcal{D}}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} \ell(f_{\theta}(x), y)$. Low empirical risk alone does not establish low population risk [Shalev-Shwartz and Ben-David, 2014, Secs. 2.1–2.3]. For a differentiable objective $\widehat{\mathcal{L}}$, a gradient step is $\theta_{t+1} = \theta_t - \eta_t \nabla \widehat{\mathcal{L}}(\theta_t)$, with step size η_t ; this expression supplies no unconditional convergence guarantee. Hebbian and Oja rules are related learning examples, not biological equivalents of every optimizer [Hebb, 1949, Oja, 1982].

Evolon (Evo). A schematic generation is $P_{t+1} = \text{Replace}(P_t, \text{Vary}(\text{Select}(P_t)))$; selection, variation, and replacement differ across algorithms. The Price identity is

$$\Delta \bar{z} = \frac{\text{Cov}(w, z) + \mathbb{E}[w \Delta z]}{\bar{w}}, \quad \bar{w} > 0.$$

Here z is the ancestor trait, w its descendant contribution, Δz the within-lineage change, and averages use ancestor frequencies. It decomposes change; without a model of those quantities it does not predict adaptation or guarantee fitness improvement [Price, 1970, Frank, 2012]. A separate continuous-time selection model is $\dot{p}_i = p_i(f_i - \bar{f})$, where p_i is type frequency, f_i its growth payoff and $\bar{f} = \sum_j p_j f_j$; mutation and drift require additional terms or models [Taylor and Jonker, 1978].

Symbion (Sym). Robinson’s resolution calculus supports sound inference and refutation completeness for first-order clauses [Robinson, 1965, Sec. 6]. Those properties concern that calculus and its assumptions, not every symbolic program. A bounded or unfair search can fail to find a refutation; unification by itself is not a general theorem prover.

Probion (Prb). For positive model evidence, Bayes’ rule gives $p(z \mid x) = p(x, z)/p(x)$. Variational inference chooses an approximation q by maximizing

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(x, z)] - \mathbb{E}_q[\log q(z)],$$

with $\log p(x) = \text{ELBO}(q) + \text{KL}(q \parallel p(z \mid x))$ when these quantities are well-defined [Blei et al., 2017, Sec. 2.2]. Approximation-family and optimization error remain. Bayesian accounts of perception are hypotheses and computational models; they do not establish a universal neural implementation [Knill and Pouget, 2004].

Scholon (Sch). A^* guarantees depend on the search variant, heuristic and graph assumptions; admissibility is not an unconditional efficiency guarantee [Hart et al., 1968]. For a finite discounted Markov decision process with bounded rewards and $0 \leq \gamma < 1$, model-based value iteration applies

$$V_{k+1}(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V_k(s') \right].$$

Here r is expected immediate reward and P the transition model. This is a planning update; it does not require learned parameters. Reinforcement-learning methods may instead learn values or policies from interaction, with discounted return $J(\pi) = \mathbb{E}_{\pi}[\sum_{t=0}^{\infty} \gamma^t R_{t+1}]$. Expected return, learning and planning are distinct parts of the account [Sutton and Barto, 2018, Chs. 3–4].

Controlon (Ctl). Finite-horizon LQR optimizes a quadratic state/input cost for linear dynamics, with positive-semidefinite state penalties and a positive-definite input penalty in the standard unconstrained formulation [Boyd, 2009]. A Kalman filter estimates state; it does not itself choose an actuator command. Under a correctly specified linear-Gaussian model, including the initial-state and noise assumptions, it computes a conditional-mean minimum-mean-square-error estimate [Kalman, 1960, Welch and Bishop, 2001]. OPT’s broad **Ctl** category includes estimation; a probabilistic estimator also motivates **Prb**. Record that overlap explicitly instead of inferring a closed control loop from a filter’s name. Optimal-feedback models of motor behavior remain scoped biological models [Todorov and Jordan, 2002].

Swarmon (Swm). An original-style PSO coordinate update is

$$v_{id}^{t+1} = v_{id}^t + c_1 r_{1,id}^t (p_{id}^t - x_{id}^t) + c_2 r_{2,id}^t (g_d^t - x_{id}^t), \quad x_{id}^{t+1} = x_{id}^t + v_{id}^{t+1}.$$

Here p_i is a particle’s best position, g a neighborhood/global best, r_1, r_2 uniform random factors, and c_1, c_2 attraction coefficients; implementations also specify velocity limits and boundary handling [Kennedy and Eberhart, 1995]. Inertia-weight variants modify the first term. Shared best-position information supports a collective account; a population alone does not establish evolutionary reproduction. For Ant System, $\tau_{ij}^{t+1} = \rho \tau_{ij}^t + \sum_k \Delta \tau_{ij}^{(k)}$, where ρ denotes persistence, $1 - \rho$ evaporation, and deposits follow completed tours [Dorigo et al., 1996, Eqs. 1–3]. Other variants use different conventions. Neither displayed update is a general global-optimum guarantee.

14 Conclusions

OPT proposes a concise vocabulary for mechanisms and their composition, together with records of design intent and implemented behavior. These records may help reviewers ask which assumptions, tests, and monitors are appropriate for a particular system. They complement existing governance and architecture practices, which already call for technical and lifecycle evidence.

A mechanism label does not establish that an implementation satisfies its intended guarantees. Assurance still requires evidence about assumptions, component interactions, tests, deployment conditions, and accountable human decisions. The Intent/Code distinction makes these questions explicit; its practical value remains to be evaluated.

The next research step is a comparative evaluation of classification reliability, missed-risk detection, review effort, and residual errors. Until such evidence exists, OPT should be treated as a candidate documentation and review aid rather than a validated safety control or a replacement for established assurance practice.

Acknowledgements, Contributions, and Disclosures

Wesley R. Elsberry is the sole author and steward of this manuscript and the originator of the OPT proposal. The work received no funding or institutional support. The author declares no competing interests. As the proposer, he is evaluating his own framework; no independent endorsement is implied.

OpenAI Codex assisted with source inspection, mathematical and classification review, claim-source comparison, drafting and editing, bibliography preparation, LaTeX preparation,

and automated and visual checks. Codex is not an author and does not supply independent scholarly peer review. Responsibility for the argument and publication remains with the author.

The paper reports no new human- or animal-subject study and no controlled test of OPT’s effectiveness.

A Mnemonic Labels and Their Etymological Motivations

The canonical OPT codes are **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, and **Swm**. The longer labels below are author-chosen mnemonics. Their English stems have historical etymologies, but those histories do not establish the meanings or validity of the proposed computational categories. The ending *-on* is used by analogy with familiar scientific names; no common historical morphology, established technical usage, or priority of coinage is claimed.

Lrn — *Learnon*. The label combines English *learn* with *-on*. The dictionary traces *learn* through Middle English to Old English *leornian* [Merriam-Webster, n.d.c]. In OPT it suggests parameter adaptation within a model.

Evo — *Evolon*. The label is a shortened blend inspired by *evolution*. That word’s history includes Latin *evolutio*, referring to unrolling, and later senses of unfolding and development [Merriam-Webster, n.d.b]. OPT associates the mnemonic with population change through variation, inheritance, and selection.

Sym — *Symon*. The stem recalls *symbol*, whose history includes Latin *symbolum* and Greek *symbolon*, a sign or token [Merriam-Webster, n.d.g]. In OPT the label suggests explicit rule application and symbolic inference; a symbolic representation alone does not establish **Sym**.

Prb — *Probion*. The spelling is an author-chosen mnemonic inspired by *probability*. The related English adjective *probable* derives from Latin *probabilis*, through a verb concerning testing or approval [Merriam-Webster, n.d.d]. The mnemonic denotes inference over represented probabilities. Randomness alone does not establish **Prb**, and probabilistic inference may coexist with learning or search.

Sch — *Scholon*. This label makes a loose mnemonic association with *school* and study. The history of *school* reaches Greek *scholē*, with senses including leisure, discussion, and a place of instruction [Merriam-Webster, n.d.e]. It is not a historical derivation of the computational term *search*. OPT uses the association only to suggest exploration of alternatives, including continuous relaxation over candidate solution states.

Ctl — *Controlon*. The label extends English *control*. Its history includes Anglo-French *contrerouler*, an audit-related verb, and ultimately Medieval Latin *contrarotulus*, a counter-register [Merriam-Webster, n.d.a]. OPT uses **Ctl** for feedback regulation and, in its broad sense, state estimation. Estimation can overlap **Prb**; the component and lifecycle phase must be specified.

Swm — *Swarmon*. The label combines *swarm* with *-on*. The English noun traces through Middle English to Old English *swearm* [Merriam-Webster, n.d.f]. The OPT mnemonic refers to collective operation through interacting units. A population alone is insufficient: the operative interactions matter.

These labels are aids to exposition. Classification depends on the operational definitions and evidence associated with the canonical codes, not on word roots.

B OPT-Code v1.0: Naming Convention

Purpose. Provide compact, semantically transparent names that self-identify an AI system’s operative mechanism(s). These are the *only* public OPT names; legacy signal types remain descriptive but are not taxonomic.

Roots (frozen set in v1.0)

Short	Name	Mechanism
Lrn	Learnon	Parametric learning (loss/likelihood/return)
Evo	Evolon	Population adaptation (variation/selection/inheritance)
Sym	Symbion	Symbolic inference (rules/constraints/proofs)
Prb	Probion	Probabilistic inference (posteriors/ELBO)
Sch	Scholon	Search, planning, solution-state optimization
Ctl	Controlon	Control & estimation (feedback/Kalman/LQR/MPC)
Swm	Swarmon	Collective/swarm (stigmergy/distributed rules)

Sch includes continuous constraint relaxation over candidate solution states, as in the fixed-weight Hopfield–Tank solving phase. Coefficient configuration alone is not **Lrn**. Optimization alone does not justify adding **Sch** to a learning, evolutionary, or swarm mechanism.

Composition syntax

- A+B: co-operative mechanisms (e.g., **LRN+SCH**).
- A/B: hierarchical nesting, outer/inner (e.g., **EVO/LRN**).
- A{B,C}: parallel ensemble (e.g., **SYM{LRN,PRB}**).
- [A→B]: sequential pipeline (e.g., **[LRN→CTL]**).

Attributes (orthogonal descriptors)

Optional, mechanism-agnostic, appended after a semicolon:

OPT=Evo/Lrn+Ctl; Rep=param; Obj=fitness; Data=sim; Time=gen; Human=low

Keys: Rep (representation), Locus, Obj, Data, Time, Human.

Grammar reference

Appendix C gives the proposed grammar, including pipelines, ensembles, and the evidence field.

Stability and change control

S1 (Root freeze). The seven roots above are fixed for this proposed version. **S2 (Extensions via attributes).** New nuance is expressed via attributes, not new roots. **S3 (Mechanism distinctness).** Proposals to add a root in a future major version must prove a distinct operational mechanism not subsumable by existing roots. **S4 (Compatibility).** Parsers may accept legacy aliases but must render short names only. **S5 (Priority).** Classification changes require evidence and version history; first publication does not confer correctness.

C Proposed Grammar of OPT-Code and OPT-Intent

The following EBNF grammar defines the proposed syntax used in the examples. Conformance by an existing tool must be checked against this specification.

```
Code      ::= "OPT=" Expr (";" Field)*
Intent    ::= "INTENT-OPT=" Expr (";" IntentField)*
Expr      ::= Nested ("+" Nested)*
Nested    ::= Primary ("/" Primary)*
Primary   ::= Root | "(" Expr ")"
           | "[" Expr "->" Expr ("->" Expr)* "]"
           | Root "{" Expr ("," Expr)+ "}"
Root      ::= "Lrn" | "Evo" | "Sym" | "Prb"
           | "Sch" | "Ctl" | "Swm"
Field     ::= Key "=" Value
Key       ::= "Rep" | "Locus" | "Obj" | "Data"
           | "Time" | "Human"
IntentField ::= IntentKey "=" Value
IntentKey ::= "INTENT-GOAL" | "INTENT-CONSTRAINTS"
           | "INTENT-RISKS" | "INTENT-EVIDENCE"
           | "INTENT-CONTEXT"
```

Value is a nonempty string without semicolons or line breaks. Horizontal whitespace around syntax tokens is ignored. A displayed arrow is typeset shorthand for the ASCII token `->`. Multi-line examples of Intent fields are display forms of a single semicolon-separated record.

Composition semantics. $A+B$ denotes cooperating components; A/B denotes outer/inner nesting; $A\{B,C\}$ denotes a coordinating root with parallel components; and $[A\rightarrow B]$ denotes a directed pipeline. Nesting binds more tightly than cooperation. Unparenthesized nesting chains associate to the right: $A/B/C$ means $A/(B/C)$. Parentheses make a different grouping explicit. These descriptions require a component map and lifecycle scope; syntax alone cannot establish mechanism correctness.

Abstention. The prompts' `OPT=Unknown` output is an abstention marker outside the grammar, not an eighth root or a valid classified record. Store the reason and route it for review.

Versioning and extension. Record the grammar revision alongside each artifact. Unknown fields must be reported and retained for review rather than silently discarded in a release decision. Classification records should preserve their evidence and version history; a later classification may be better supported than the first published assignment.

C.1 Evaluation Protocol for OPT-Code Classification

The following proposed protocol specifies records needed for a reproducible and auditable study of evaluating OPT-Code classifications generated by large language models. The protocol aligns with reproducible computational research practices and is designed to support reliable inter-model comparison, adjudication, and longitudinal quality assurance.

C.1.1 Inputs

For each system under evaluation, the following inputs are provided:

1. **System description:** source code, algorithmic description, or detailed project summary.
2. **Candidate OPT-Code:** produced by a model using the minimal or maximal prompt (Section D.2).
3. **Candidate rationale:** a short explanation provided by the model describing its classification.

These inputs are then supplied to the OPT-Code Prompt Evaluator (Appendix D.3).

C.1.2 Evaluation Pass

The evaluator produces:

- **Verdict:** PASS, WEAK_PASS, or FAIL.
- **Score:** an integer from 0–100.
- **Issue categories:** format, mechanism, parallelism/pipelines, composition, and attribute plausibility.
- **Summary:** a short free-text evaluation.

The legacy threshold of 70 is a proposed triage setting, not a calibrated probability or validated acceptance criterion. It may route a candidate to review but must not authorize a consequential release. Report sensitivity to this threshold and compare with an independently adjudicated reference.

C.1.3 Double-Annotation Procedure

To expose some disagreements, without assuming independent errors, each system description is classified independently by two LLMs or by two runs of the same LLM with different seeds:

1. Model A produces an OPT-Code and rationale.
2. Model B produces an OPT-Code and rationale.
3. Each is independently evaluated by the Prompt Evaluator.

Inter-model agreement is quantified using one or more of the following metrics:

- **Exact-match OPT** (binary): whether the root composition matches identically.

- **Partial-match similarity:** Jaccard similarity between root sets (e.g., comparing Evo+Lrn with Evo+Sch).
- **Levenshtein distance** (string distance over the structured OPT–Code line).
- **Weighted mechanism agreement:** preregistered, justified weights reflecting hypothesized semantic distances between roots (e.g., **Swm** is closer to **Evo** than to **Sym**).

Discrepancies trigger a joint review.

C.1.4 Adjudication Phase

If the two candidate classifications differ substantially (e.g., different root sets or different compositions), an adjudication step is performed:

1. Provide the system description, both candidate OPT–Codes, both rationales, and both evaluator reports to a third model (or human expert).
2. Use a specialized *adjudicator prompt* that asks the model to choose the better classification according to OPT rules.
3. Require the adjudicator to justify its decision and to propose a final, consensus OPT–Code.

A new evaluator pass checks consistency with the rubric. It cannot confirm correctness independently; the evidence and remaining disagreement require human adjudication.

C.1.5 Quality Metrics

The following quality-reporting metrics may be computed at the level of a batch of evaluations:

- **Evaluator pass rate:** proportion of PASS or WEAK_PASS verdicts.
- **Inter-model consensus rate:** exact match vs. non-exact match.
- **Root-level confusion matrix:** which OPT roots are mistaken for others, across models or datasets.
- **Pipeline sensitivity:** how often parallelism or data pipelines are misclassified as mechanisms.

These metrics allow the OPT framework to be applied consistently and help identify systematic weaknesses in model-based classification pipelines.

C.1.6 Longitudinal Tracking

For large-scale use (e.g., benchmarking industrial systems), it is recommended to store for each case:

- the system description,
- both model classifications,
- evaluator verdicts and scores,

Algorithm 1 OPT–Code Evaluation and Adjudication Pipeline

Require: System description S

```
1:  $C_A \leftarrow \text{ClassifierModel}(S, \text{MinimalPrompt or MaximalPrompt})$ 
2:  $C_B \leftarrow \text{ClassifierModel}(S, \text{MinimalPrompt or MaximalPrompt})$ 
3:  $E_A \leftarrow \text{EvaluatorModel}(S, C_A, \text{EvaluatorPrompt})$ 
4:  $E_B \leftarrow \text{EvaluatorModel}(S, C_B, \text{EvaluatorPrompt})$ 
5: if ( $E_A.\text{verdict}$  and  $E_B.\text{verdict}$ ) are acceptable then
6:   if  $C_A.\text{OPT} = C_B.\text{OPT}$  then
7:     return  $C_A$  as candidate OPT–Code for human review
8:   else
9:      $J \leftarrow \text{AdjudicatorModel}(S, C_A, E_A, C_B, E_B, \text{AdjudicatorPrompt})$ 
10:     $C^* \leftarrow J.\text{Final OPT–Code}$ 
11:     $E^* \leftarrow \text{EvaluatorModel}(S, C^*, \text{EvaluatorPrompt})$ 
12:    if  $E^*.\text{verdict}$  acceptable then
13:      return  $C^*$  as candidate OPT–Code for human review
14:    else
15:      Flag case for human review
16:    end if
17:  end if
18: else
19:   Flag case for human review
20: end if
```

- adjudicated decisions,
- timestamps and model versions.

Such archival enables longitudinal analysis of model performance, drift, and taxonomy usage over time.

C.2 Evaluation Workflow Pseudocode

D Appendix: OPT–Code Prompt Specifications

This appendix collects proposed prompt formulations for eliciting OPT–Code classifications from large language models and to evaluate those classifications for consistency with a rubric. They have not been validated as correctness or release tests; model approval remains a candidate judgment.

D.1 Minimal OPT–Code Classification Prompt

The minimal prompt is designed for inference-time use and lightweight tagging pipelines. It assumes a basic familiarity with the OPT roots and emphasizes mechanism-based classification over surface labels.

You are an analyzer that assigns an OPT-Code to AI systems based on the
↪ system's
operative mechanism.

OPT roots (mechanism classes):

- Lrn (Learning): parametric updates within a fixed model; gradients,
 - ↪ Hebbian/Oja,
 - TD learning, policy/value updates.
- Evo (Evolutionary): population-based variation + selection + inheritance;
 - ↪ GA,
 - ES, GP, neuroevolution, clonal selection.
- Sym (Symbolic/Logic/Rules): explicit symbolic structures, unification, rule application, theorem proving, production systems, structured planning.
- Prb (Probabilistic): explicit probabilistic models and inference; Bayesian
 - ↪ nets,
 - HMMs, graphical models, probabilistic programming, VI, MCMC.
- Sch (Search/Planning/Optimization): search over discrete/continuous solution states; A*, MCTS, branch-and-bound, continuous constraint relaxation. Fixed-weight Hopfield-Tank solving is Sch. Coefficient configuration alone is not Lrn. Optimization alone does not add Sch to Lrn, Evo, or Swm.
- Ctl (Control/Estimation): feedback control and state estimation; PID, LQR, MPC, trajectory regulation. State estimators require scoped treatment.
- Swm (Swarm/Multi-agent Local Rules): many simple agents with local
 - ↪ interactions
 - or neighborhood rules; PSO, ACO, boids, immune networks.

Rules:

- Focus strictly on the mechanism: update rules, iteration structure, and data flow that produces behavior. Ignore task domain and surface labels like
 - ↪ "AI."
- Parallelism & pipelines:
 - DO NOT treat threads, actors, async/await, CUDA kernels, batching, distributed jobs, or multi-stage data pipelines as OPT mechanisms.
 - Parallelism counts ONLY when the algorithmic core uses many interacting
 - ↪ local
 - agents (Swm), population-level adaptation (Evo), or true multi-branch
 - ↪ search (Sch).
 - Pipelines are NOT mechanisms; use [A->B] only for stages with distinct
 - ↪ operative mechanisms.
- Composition:
 - Use "X+Y" when roots operate together in the same core loop.
 - Use "X/Y" for outer/inner nesting, not sequential phases.
 - Use "[X->Y]" for pipelines and "X{Y,Z}" for coordinated ensembles.
- Record training versus inference. Randomness is not enough for Prb; a population is not enough for Swm or Evo. Do not infer learning from neural architecture alone. Flag ambiguous root boundaries for human review.
- Follow the supplied OPT grammar; record lifecycle scope and evidence.

Also assign orthogonal attributes:

- Rep: representation (bitstring, rules, graph, NN-weights, agent-state, ↪ etc.).
- Obj: objective (loss, reward, likelihood, constraint-satisfaction, cost).
- Data: data regime (labels, unlabeled, self-play, environment, expert demos, ↪ signals).
- Time: adaptation timescale (online, offline, episodic, generations).
- Human: human involvement (low/medium/high).

Output format:

- 1) OPT=<root(s)>; Rep=<...>; Obj=<...>; Data=<...>; Time=<...>; Human=<...>
- 2) Rationale: 2--4 sentences explaining the mechanism and classification.

If insufficient data:

OPT=Unknown; Rep=?; Obj=?; Data=?; Time=?; Human=?
and explain what information is missing.

D.2 Maximal Expert OPT-Code Classification Prompt

The maximal prompt elaborates all root definitions, clarifies the treatment of parallelism and pipelines, and details rules for composition. It is a proposed research prompt, not validated for fine-tuning or consequential release decisions.

You are an expert mechanism analyst. Your task is to assign an OPT-Code to a system based solely on the operative computational mechanisms present in
↪ either:

- source code, or
- a system/project description.

You must ignore marketing language, domain labels, or incidental engineering choices. You must classify only the underlying algorithmic mechanism.

=====

OPT ROOTS: DEFINITIONS

=====

Lrn (Learning)

Parametric updating within a fixed architecture: gradient descent, Adam, Hebbian/Oja rules, predictive coding, error-backprop, RL policy/value updates, TD(lambda), actor-critic.

Evo (Evolutionary)

Population-based mechanisms involving variation, selection, inheritance, and reproduction. Examples: GA, ES, GP, CMA-ES, neuroevolution, clonal selection, immune-inspired evolutionary search.

Sym (Symbolic / Logic / Rules)

Manipulation of explicit symbolic structures: logic rules, constraints, production systems, theorem proving, STRIPS-style planning,
↪ forward/backward
chaining, unification, rule-based expert systems.

Prb (Probabilistic)

Computation expressed as uncertainty propagation or probabilistic

↪ inference:

Bayesian networks, HMMs, CRFs, factor graphs, particle filters, MCMC, variational inference, probabilistic programming.

Sch (Search / Planning / Optimization)

Search over discrete or continuous candidate solution states: A*, IDA*, branch-and-bound, MCTS, continuous constraint relaxation. Fixed-weight Hopfield-Tank solving is Sch: states change under an encoded objective. Coefficient configuration alone is not Lrn; identify an actual learning phase separately, offline or online. An optimization objective alone does not add Sch to Lrn, Evo, or Swm. Composition requires distinct
↪ mechanisms.

Ctl (Control / Estimation)

Feedback regulation, trajectory tracking, or state estimation: PID, LQR, MPC. Kalman estimators overlap Prb and the broad Ctl definition. Key
↪ signature:
closed-loop feedback and an explicit control objective.

Swm (Swarm / Multi-agent Local Rules)

Many simple agents with local interactions: cellular automata, boids, ant-colony optimization, PSO, immune-network models, distributed
↪ consensus.

=====

PARALLELISM AND PIPELINES: DO NOT MISCLASSIFY

=====

Execution-level parallelism is NOT a mechanism.

Treat ALL of the following as irrelevant to OPT classification:

threads, processes, async/await, multiprocessing, queues,
CUDA/GPU kernels, tensor parallelism, model parallelism,
SIMD, vectorization, batching, map/reduce,
ETL-style pipelines (preprocess -> model -> postprocess),
ROS nodes, RPC, microservices, Spark jobs,
Kubernetes orchestration, distributed training frameworks.

Parallelism or pipelines only influence OPT when they are intrinsic to the computation itself:

Swm: many interacting agents with local rules; parallelism reflects the mechanism, not engineering.

Evo: variation, reproduction, and selection; parallel evaluation alone does not establish that mechanism.

Sch: multi-branch exploration in search trees.

Prb: particle filters with particle-wise updates count ONLY if the model semantics requires distributional representation.

Pipeline stages DO NOT justify an OPT pipeline expression unless the stages implement distinct root mechanisms (e.g., Evo -> Sym -> Prb).

HOW TO COMPOSE ROOTS

- Record training versus inference. Randomness is not enough for Prb; a population is not enough for Swm or Evo. Do not infer learning from neural architecture alone. Flag ambiguous root boundaries for human review.

Use "+" when mechanisms are tightly integrated within one core loop:
Evo+Lrn, Lrn+Sch, Swm+Prb, etc.

Use "/" for outer/inner nesting, with right-associative chains.

Use "[A->B]" for distinct sequential mechanisms.

Use "A{B,C}" for a coordinating root with parallel components.

Use parentheses to make grouping explicit. Record lifecycle stage and
↪ evidence.

ORTHOGONAL ATTRIBUTES

Rep = representation (bitstring, graph, rules, NN-weights, trajectories,
agent-state, signals, distributions)
Obj = objective (loss, reward, likelihood, energy, constraint-violation)
Data = data regime (labels, unlabeled, environment, self-play, expert demos)
Time = adaptation timescale (online, offline, generations, episodic)
Human = human involvement (high / medium / low)

OUTPUT FORMAT

- 1) OPT=<roots>; Rep=<...>; Obj=<...>; Data=<...>; Time=<...>; Human=<...>
- 2) Rationale: 3--6 sentences describing the mechanism and why these roots
↪ apply.

If information is incomplete:
OPT=Unknown; Rep=?; Obj=?; Data=?; Time=?; Human=?
Rationale: explain missing elements.

D.3 Evaluating OPT-Codes

The evaluator prompt is a meta-level specification: it assesses whether a given candidate OPT-Code and rationale respect the OPT taxonomy and associated guidelines. This enables automated or semi-automated review of classifications generated by other models or tools.

D.4 OPT-Code Evaluator Prompt

You are an OPT-Code evaluation assistant. Your job is to check whether a candidate OPT classification follows the OPT rules and is mechanistically correct.

Inputs you will be given:

- 1) System description: a code snippet or project/system description.
- 2) Candidate OPT-Code line (from another model), of the form:
OPT=<roots>; Rep=<...>; Obj=<...>; Data=<...>; Time=<...>; Human=<...>
- 3) Candidate rationale: 2--6 sentences explaining the candidate's choice.

You must evaluate the candidate against the following criteria:

- (1) Format compliance:
 - Does the candidate produce exactly one OPT= line with the correct
↪ fields?
 - Are the roots valid (Lrn, Evo, Sym, Prb, Sch, Ctl, Swm)?
 - Does the expression follow the supplied OPT grammar, including
↪ parentheses,
nesting, pipelines and ensembles?
- (2) Mechanism correctness:
 - Do the chosen roots match the operative mechanism in the system
↪ description?
 - Is there any root that is missing but clearly present?
 - Is any root included that is not supported by the description?
 - Continuous relaxation over candidate solution states qualifies as Sch;
fixed-weight Hopfield-Tank solving is an example. Coefficient
↪ configuration
alone is not Lrn. An optimization objective alone does not justify
↪ adding
Sch to Lrn, Evo, or Swm; require distinct operative mechanisms.
- (3) Parallelism and pipelines:
 - Does the candidate incorrectly treat threads, GPU kernels, async,
↪ pipelines,
or distributed infrastructure as OPT mechanisms (e.g., calling something
Swm or Sch only because it is parallel)?
 - If so, this is a serious error.
- (4) Composition correctness:
 - Use "+" only for tightly integrated mechanisms in the same core loop.
 - Use "/" for outer/inner nesting, "[A->B]" for sequential mechanisms,
and "A{B,C}" for coordinated parallel components.
 - Flag composition claims that lack supporting component or lifecycle
↪ evidence.
- (5) Attribute plausibility:
 - Are Rep, Obj, Data, Time, and Human reasonably consistent with the
↪ system

description?

- They do not need to be unique, but they must be defensible.

Your output must use the following structure:

Verdict: <PASS | WEAK_PASS | FAIL>

Score: <integer from 0 to 100>

Issues:

- Format: <short comment>
- Mechanism: <short comment>
- Parallelism/Pipelines: <short comment>
- Composition: <short comment>
- Attributes: <short comment>

Summary: <2--4 sentences giving an overall assessment and key corrections, if
↪ any>.

Guidelines:

- PASS means: no major errors; at most minor debatable choices.
- WEAK_PASS means: generally acceptable, but with at least one non-trivial
↪ issue
that should be corrected before publication.
- FAIL means: at least one serious misunderstanding of the mechanism, or clear
violation of the parallelism/pipeline rules, or badly wrong roots.

D.5 Storage Formats for OPT Audit Logs

For large-scale or longitudinal use, we recommend storing OPT classifications and evaluations in a machine-readable log format. Two practical options are:

JSON Lines (JSONL). Each line contains a single JSON object describing one system evaluation, including:

- system identifier and textual description,
- candidate OPT-Codes and rationales,
- evaluator verdicts, scores, and issue summaries,
- adjudicator decisions and final OPT-Code,
- timestamps, model identifiers, and prompt variants.

JSONL is convenient for streaming pipelines, command-line tools, and map-reduce processing.

YAML. YAML provides more human-friendly syntax and supports comments. It is useful for curated datasets or hand-edited corpora of OPT-annotated systems. The same fields as above can be stored in a nested structure, with separate top-level keys for description, candidates, evaluations, adjudication, and metadata.

Schema. A minimal schema for either JSONL or YAML includes:

- **id:** unique system identifier,
- **description:** text or reference to source code,
- **candidates:** list of OPT-Codes and rationales,
- **evaluations:** evaluator outputs for each candidate,
- **adjudication:** final decision and rationale (if any),
- **final:** final OPT-Code and attributes,
- **meta:** timestamps, model versions, prompt names.

Such logs support reproducibility, auditability, and downstream statistical analysis of taxonomy usage and model performance.

E Selecting Models for a Classification Study

Choose models through task-specific evaluation rather than parameter count or a general coding leaderboard alone.

For each candidate, freeze its identifier or weight revision, runtime, quantization, context limit, prompt, decoding settings, and hardware. Evaluate format validity, root and composition accuracy against adjudicated references, abstention on insufficient evidence, and robustness to misleading labels. Measure latency, resource use, and reviewer effort separately. A model that follows the format can still misclassify mechanisms.

Using the same model for classification and evaluation may repeat the same error. Different models also share training and evaluation biases. Keep disagreement visible and require accountable human review for consequential conclusions. Select a configuration only after documenting its observed trade-offs on the intended task; this paper supplies no benchmark-based recommendation of a current model family.

F OPT-Aware Agent Skills

This appendix enumerates conceptual skills that may be incorporated into agentic AI systems to support mechanism-aware planning, execution, diagnosis, and repair using the Operational Premise Taxonomy (OPT). These skills are descriptive rather than prescriptive; they define *capabilities* that may be implemented using a variety of agent architectures, programming languages, or reasoning engines.

F.1 OPT Classification Skill

Purpose. Identify the operative mechanisms present in a system description, source code, tool invocation, or execution trace.

Description. Given a representation of a system or subcomponent, the agent produces an OPT-Code describing the dominant and supporting mechanisms. This skill enables mechanism awareness and provides the foundation for subsequent reasoning.

F.2 OPT–Intent Proposal Skill

Purpose. Generate an OPT–Intent declaration during design or planning.

Description. Given a goal, constraints, and deployment context, the agent proposes a set of intended operative mechanisms, anticipated risks, and constraints. This skill supports disciplined planning and avoids unexamined default choices.

F.3 OPT Alignment Evaluation Skill

Purpose. Assess consistency between OPT–Intent and OPT–Code.

Description. The agent compares intended mechanisms against those actually employed, identifying additions, omissions, or substitutions. Deviations are flagged as mechanism drift and may trigger repair or escalation.

F.4 Mechanism-Guided Error Diagnosis Skill

Purpose. Interpret errors in terms of underlying operative mechanisms.

Description. Rather than diagnosing failures solely at the level of outputs, the agent conditions its diagnosis on the OPT root(s) involved, narrowing the space of plausible failure modes and repair strategies.

F.5 Constraint-Preserving Repair Skill

Purpose. Propose repairs that respect declared mechanism constraints.

Description. Given an error and an OPT–Intent declaration, the agent proposes corrective actions that preserve or explicitly justify changes to the operative mechanisms.

F.6 OPT Memory and Traceability Skill

Purpose. Maintain a mechanism-aware record of decisions and outcomes.

Description. The agent records OPT–Intent declarations, OPT–Code observations, alignment checks, and repairs, supporting auditability and longitudinal analysis.

OPT Roots and Characteristic Agent Failure Modes

OPT Root	Primary Mechanism	Characteristic Failure Modes
Lrn	Parametric learning	Overfitting, distributional shift, catastrophic forgetting, spurious correlations
Evo	Evolutionary adaptation	Premature convergence, loss of diversity, unstable fitness dynamics
Sym	Symbolic reasoning	Rule inconsistency, brittleness, combinatorial rule explosion
Prb	Probabilistic inference	Miscalibration, uncertainty collapse, incorrect priors
Sch	Search and optimization	Heuristic bias, local minima, combinatorial explosion
Ctl	Control and feedback	Oscillation, instability, delayed response, unsafe transients
Swm	Swarm dynamics	Incoherent emergence, sensitivity to noise, coordination failure

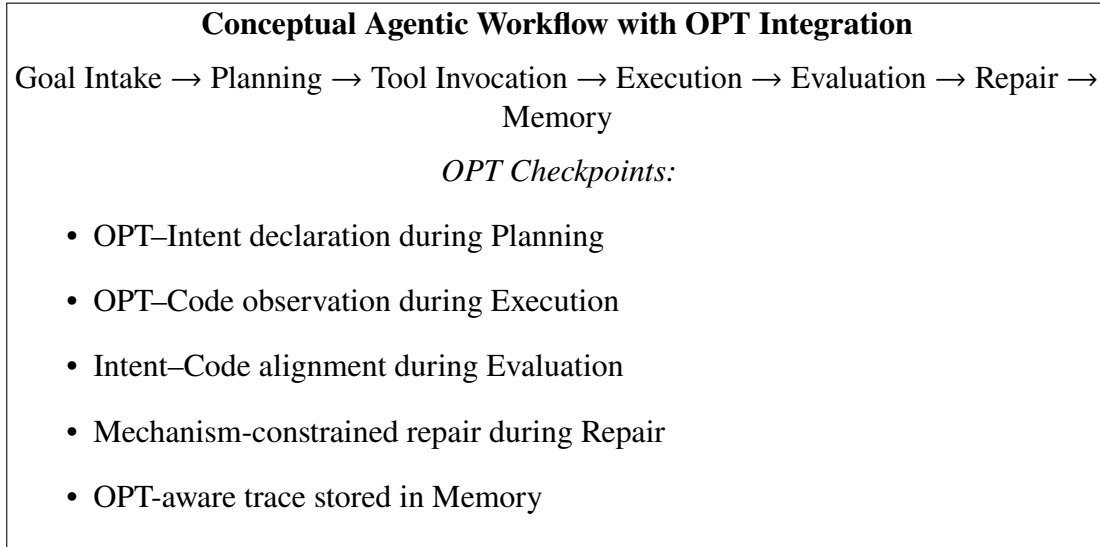


Figure 3: Agentic AI workflow annotated with OPT checkpoints. OPT operates as a mechanism-aware abstraction layer that augments planning, diagnosis, repair, and governance without prescribing a specific agent architecture.

G Conceptual Example: OPT in an Agentic Development Workflow

To illustrate the practical role of OPT in agentic AI systems, we consider a particular scenario: an autonomous development agent tasked with constructing and maintaining a production scheduling system under dynamic constraints.

G.1 Step 1: Goal Intake and OPT–Intent Declaration

The agent receives the following high-level goal:

Design a system to optimize production schedules under variable supply, equipment downtime, and priority constraints.

The agent proposes the following OPT–Intent:

```
INTENT-OPT = Sch/Sym
INTENT-GOAL = minimize-production-delay
INTENT-CONSTRAINTS = deterministic, explainable, real-time
INTENT-RISKS = combinatorial-explosion
```

The agent explicitly selects search (**Sch**) for combinatorial optimization and symbolic reasoning (**Sym**) for constraint enforcement, while excluding learned predictors as an explicit design constraint. Determinism and explainability still require separate evidence; they do not follow from excluding learning.

G.2 Step 2: Implementation and OPT–Code Observation

During implementation, the agent integrates:

- A heuristic search planner,
- A rule-based constraint validator,
- A neural network model for predicting machine failure.

The resulting OPT–Code is:

```
OPT = Sch/Sym/Lrn;
Rep = permutations + rules + predictive-model;
Time = iterative + online-adjust
```

G.3 Step 3: Drift Detection

Comparison with OPT–Intent reveals mechanism drift:

- **Lrn** was introduced,
- The new predictor requires review against the stated constraints,
- Risk profile has changed.

The agent flags this deviation and evaluates whether predictive learning violates declared governance constraints.

G.4 Step 4: Mechanism-Guided Error

Suppose the system exhibits unstable schedules under rare supply patterns. Given the OPT–Code, the agent treats distributional shift in the learned predictor as one hypothesis. It must also test constraints, inputs, scheduling heuristics, and interfaces before attributing the failure.

G.5 Step 5: Constraint-Preserving Repair

The agent proposes two alternatives:

1. Replace the neural predictor with symbolic failure rules (**Sym**),
2. Retain **Lrn** but update OPT–Intent and governance constraints.

The first option preserves the original intent. The second requires explicit authorization.

G.6 Step 6: Reclassification and Repair Testing

If the symbolic replacement is adopted, the new OPT–Code becomes:

$$\text{OPT} = \text{Sch/Sym}$$

The declared roots now match the original intent. Repair is verified only after implementation inspection and tests of the scheduling requirements; a matching label alone does not show that instability has been fixed.

G.7 Discussion

This example illustrates how OPT provides:

- Mechanism-aware planning,
- Explicit drift detection,
- Targeted error diagnosis,
- Governance-compatible repair,
- Structured traceability.

Importantly, OPT does not constrain the agent’s architecture. Instead, it provides a stable abstraction layer that connects design commitments, implementation choices, and remediation strategies.

H Proposed Tool Support for OPT-Aware Agentic Systems

The capabilities below are design requirements. Their listing does not establish implemented or validated tooling.

While OPT is fundamentally a conceptual taxonomy, its intended uses could be supported by tooling for classification, verification, and alignment analysis. In agentic AI systems, such tooling is proposed to enable partial automation of mechanism- aware reasoning and governance.

H.1 OPT Classification and Verification Tools

Automated classifiers may infer OPT–Code from source code, architectural descriptions, or execution traces. Verification tools can then assess syntactic validity, semantic consistency, and completeness of OPT–Code expressions. Such tools would need validation for both static analysis and runtime introspection.

H.2 OPT–Intent and Alignment Evaluation

OPT–Intent declarations provide a reference against which agent behavior can be evaluated. Tooling that compares OPT–Intent with observed OPT–Code is intended to help reviewers detect mechanism drift and unplanned changes in operative premises. Long-running or self-modifying agentic systems are candidate evaluation settings.

H.3 LLM-Supported Reasoning

Large language models can assist in OPT classification, intent proposal, and alignment evaluation when guided by structured prompts. OPT prompts ask these models to reason explicitly about operative mechanisms. Whether this reduces category errors or unexamined defaults remains untested.

H.4 Integration into Agentic Workflows

OPT-aware tools may be invoked as part of planning, evaluation, or repair phases in agentic workflows. By exposing mechanism-level information to the agent, these tools are intended to support planning, remediation, and reporting; those benefits require comparative evaluation.

H.5 Governance and Auditability

Proposed OPT tooling would support governance by producing durable, machine-readable records of mechanism choices and changes over time. These records can be used for internal review or external audit, but are not by themselves evidence of regulatory compliance. Claims about implemented mechanisms still require adequate access to technical evidence, including relevant proprietary details.

I Claim, Figure, and Evidence Provenance

Stewardship and time. Wesley R. Elsberry is the responsible author and steward. The evidence notes were prepared with OpenAI Codex on 26 September 2026. Confidence below is a qualitative assessment of claim–evidence fit, not a calibrated probability.

P1: Proposed taxonomy and notation. The seven roots, composition syntax, Intent/Code distinction, and use examples are author-designed proposals. Their specification permits inspection and testing; completeness, minimality, and improved outcomes remain unestablished. Conformance of a tool to the proposed grammar requires an implementation check.

P2: Framework comparisons. NIST AI RMF sections MAP 2 and MAP 4, OECD’s AI-model discussion (pp. 44–46), the EU AI Act recital 12 and Annex IV, Model Cards’ abstract, and Nygard’s ADR essay support the framework descriptions. Public ISO scope pages support only scope-level descriptions. Confidence is high for the cited positive features within those access limits. No exhaustive survey of all alternatives was performed.

P3: Historical failures. The Ariane, Mars Climate Orbiter, Knight Capital, MCAS, blackout, flash-crash, and Google Flu Trends examples are literature illustrations with their cited sources. They are not a sampled dataset, an OPT intervention, or proof that OPT would have prevented the incidents. The proposed OPT interpretations are explanatory hypotheses rather than causal findings. Source checking for this paper does not constitute a complete full-text audit of every incident report.

P4: Mathematical and biological correspondences. Equations and qualitative mappings summarize the cited methodological literature. Their assumptions and scope govern any associated guarantee. The biological analogies do not prove common implementation or transfer guarantees between systems. Source inspection covers selected objectives, inference bounds, calculus properties, and algorithm descriptions; classic-system sources support specified operations rather than complete implementation audits. The glossary cites dictionary word histories for its stems and identifies the longer labels as author-chosen mnemonics. The Hopfield–Tank solving phase is assigned **Sch** under the proposed continuous-relaxation inclusion rule. Its neural dynamics are source-supported; the OPT assignment is an author-defined interpretation, not a classification supplied by the source. Other unresolved classifications identify limitations of the current coding rules. Confidence in a source-supported operation does not establish confidence in a taxonomy-wide assignment or coverage claim.

P5: Figures, tables, and numerical statements. Figures 1 and 2 are author-designed conceptual diagrams; connections illustrate selected compositions rather than observed frequency or historical ancestry. Figure 3 is a proposed workflow. The comparison and risk tables are qualitative synthesis, not measured rankings. The 0–100 evaluator rubric and threshold of 70 in the prompt appendices are unvalidated design settings, not performance data or probabilities. No new empirical performance measurements are reported.

References

- David H. Ackley, Geoffrey E. Hinton, and Terrence J. Sejnowski. A learning algorithm for boltzmann machines. *Cognitive Science*, 9:147–169, 1985. doi:[10.1207/s15516709cog0901_7](https://doi.org/10.1207/s15516709cog0901_7). URL https://www.cs.cmu.edu/~bhiksha/courses/deeplearning/Fall.2016/pdfs/Ackley_Hinton_Sejnowski.pdf.
- Jesus Alcala-Fdez and Jose M. Alonso. A Survey of Fuzzy Systems Software: Taxonomy, Current Research Trends and Prospects. *IEEE Trans. Fuzzy Syst.*, 24(1):40–56, February 2016. ISSN 1063-6706. doi:[10.1109/TFUZZ.2015.2426212](https://doi.org/10.1109/TFUZZ.2015.2426212).
- Ariane 501 Inquiry Board. Ariane 5 flight 501 failure: Report by the inquiry board. Technical report, European Space Agency, 1996. URL <https://www-users.cse.umn.edu/~arnold/disasters/ariane5rep.html>.
- David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017. doi:[10.1080/01621459.2017.1285773](https://doi.org/10.1080/01621459.2017.1285773). URL <https://www.cs.columbia.edu/~blei/fogm/2018F/materials/BleiKucukelbirMcAuliffe2017.pdf>.

- Avrim L. Blum and Merrick L. Furst. Fast planning through planning graph analysis. *Artificial Intelligence*, 90:281–300, 1997. URL <https://www.cs.cmu.edu/afs/cs/usr/avrim/www/Papers/graphplan.pdf>.
- Stephen Boyd. Linear quadratic regulator: Discrete-time finite horizon. Stanford EE363, Lecture 1, Winter 2008–09, 2009. URL <https://ee363.stanford.edu/archive/lectures/dlqr.pdf>. Official lecture notes; cost and assumptions, slides 1-2 to 1-4 inspected.
- Gail A. Carpenter and Stephen Grossberg. A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, 37: 54–115, 1987a. URL <https://sites.bu.edu/steveg/files/2016/06/CarGro1987CVGIP.pdf>.
- Gail A. Carpenter and Stephen Grossberg. ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, 26:4919–4930, 1987b. URL <https://sites.bu.edu/steveg/files/2016/06/CarGro1987AppliedOptics.pdf>.
- Leandro Nunes de Castro and Fernando J. Von Zuben. Learning and optimization using the clonal selection principle. *IEEE Transactions on Evolutionary Computation*, 6(3):239–251, 2002. doi:10.1109/TEVC.2002.1011539. URL <https://www.cs.unm.edu/~forrest/classes/immuno-class/readings/DeCastro.pdf>.
- Marco Dorigo, Vittorio Maniezzo, and Alberto Colnari. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1): 29–41, 1996. doi:10.1109/3477.484436. URL <https://jmvidal.cse.sc.edu/library/dorigo96a.pdf>. Author manuscript; update equations 1–3 inspected.
- European Union. Regulation (eu) 2024/1689 of the european parliament and of the council on artificial intelligence (ai act). Official Journal of the European Union, 12 July 2024, 2024. URL <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971. doi:10.1016/0004-3702(71)90010-5.
- Stephanie Forrest, Alan S. Perelson, Lawrence Allen, and Rajesh Cherukuri. Self-nonself discrimination in a computer. In *Proceedings of the 1994 IEEE Computer Society Symposium on Research in Security and Privacy*, pages 202–212, 1994. doi:10.1109/RISP.1994.296580. URL <https://conferences.computer.org/sp/pdfs/sp/1994/00044264.pdf>.
- Steven A. Frank. Natural selection. iv. the price equation. *Journal of Evolutionary Biology*, 25: 1002–1019, 2012. doi:10.1111/j.1420-9101.2012.02498.x. URL <https://stevefrank.org/reprints-pdf/13NatSelSeries.pdf>. Author-hosted collected series; article IV, Eq. 2 and discussion inspected.
- Julie Greensmith and Uwe Aickelin. The deterministic dendritic cell algorithm. ICARIS 2008; author manuscript deposited as arXiv:1006.1512 in 2010, 2008. URL <https://arxiv.org/abs/1006.1512>. Section 3 and Algorithm 1 inspected.
- Stephen Grossberg. The adaptive self-organization of serial order in behavior: Speech, language, and motor control. In E. C. Schwab and H. C. Nusbaum, editors, *Pattern Recognition by Humans and Machines*, volume 1, pages 187–294. Academic Press, 1986. URL <https://sites.bu.edu/steveg/files/2016/06/Gro1986SchwabNusbaum.pdf>.

- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:[10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136). URL <https://www.cs.auckland.ac.nz/courses/compsci709s2c/resources/Mike.d/astarNilsson.pdf>.
- Donald Olding Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York, 1949. URL https://books.google.com/books/about/The_Organization_of_Behavior.html?id=dZ0eDiLTwuEC.
- J. J. Hopfield and D. W. Tank. “Neural” computation of decisions in optimization problems. *Biological Cybernetics*, 52:141–152, 1985. doi:[10.1007/BF00339943](https://doi.org/10.1007/BF00339943). Primary full text inspected, pp. 144, 146–147, 149–150; fixed-weight neural constraint relaxation.
- ISO/IEC. Information technology — artificial intelligence — concepts and terminology. Technical Report ISO/IEC 22989:2022, ISO/IEC, 2022a. URL <https://www.iso.org/standard/74296.html>.
- ISO/IEC. Framework for artificial intelligence (ai) systems using machine learning (ml). Technical Report ISO/IEC 23053:2022, ISO/IEC, 2022b. URL <https://www.iso.org/standard/74438.html>.
- R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. *J. Basic Eng.*, 82(1):35–45, March 1960. ISSN 0021-9223. doi:[10.1115/1.3662552](https://doi.org/10.1115/1.3662552).
- James M. Keller, Derong Liu, and David B. Fogel. *Fundamentals of Computational Intelligence: Neural Networks, Fuzzy Systems, and Evolutionary Computation*. Wiley-IEEE Press, 2016. ISBN 9781119214342. URL <https://www.wiley-vch.de/en/areas-interest/engineering/fundamentals-of-computational-intelligence-978-1-119-21434-2>. Publisher metadata inspected 26 September 2026.
- James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN’95: International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi:[10.1109/ICNN.1995.488968](https://doi.org/10.1109/ICNN.1995.488968). URL <https://staff.washington.edu/paymana/swarm/kennedy95-ijcnn.pdf>. Scanned primary paper inspected using OCR; algorithm section checked against page image.
- David C. Knill and Alexandre Pouget. The Bayesian brain: the role of uncertainty in neural coding and computation. *Trends Neurosci.*, 27(12):712–719, December 2004. ISSN 0166-2236. doi:[10.1016/j.tins.2004.10.007](https://doi.org/10.1016/j.tins.2004.10.007).
- David Lazer, Ryan Kennedy, Gary King, and Alessandro Vespignani. The parable of google flu: Traps in big data analysis. *Science*, 343(6176):1203–1205, 2014. doi:[10.1126/science.1248506](https://doi.org/10.1126/science.1248506).
- Mars Climate Orbiter Mishap Investigation Board. Mars climate orbiter mishap investigation board phase i report. Technical report, NASA, 1999. URL https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf.
- John McDermott. R1: An expert in the computer systems domain. In *Proceedings of AAAI-80*, 1980. URL <https://cdn.aaai.org/AAAI/1980/AAAI80-076.pdf>.
- Merriam-Webster. Control: Word history. Merriam-Webster.com Dictionary, n.d.a. URL <https://www.merriam-webster.com/dictionary/control>. Accessed 26 September 2026.

- Merriam-Webster. Evolution: Word history. Merriam-Webster.com Dictionary, n.d.b. URL <https://www.merriam-webster.com/dictionary/evolution>. Accessed 26 September 2026.
- Merriam-Webster. Learn: Word history. Merriam-Webster.com Dictionary, n.d.c. URL <https://www.merriam-webster.com/dictionary/learn>. Accessed 26 September 2026.
- Merriam-Webster. Probable: Word history. Merriam-Webster.com Dictionary, n.d.d. URL <https://www.merriam-webster.com/dictionary/probable>. Accessed 26 September 2026.
- Merriam-Webster. School: Word history. Merriam-Webster.com Dictionary, n.d.e. URL <https://www.merriam-webster.com/dictionary/school>. Accessed 26 September 2026.
- Merriam-Webster. Swarm: Word history. Merriam-Webster.com Dictionary, n.d.f. URL <https://www.merriam-webster.com/dictionary/swarm>. Accessed 26 September 2026.
- Merriam-Webster. Symbol: Word history. Merriam-Webster.com Dictionary, n.d.g. URL <https://www.merriam-webster.com/dictionary/symbol>. Accessed 26 September 2026.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi:[10.1145/3287560.3287596](https://doi.org/10.1145/3287560.3287596). URL <https://arxiv.org/abs/1810.03993>. Author list and abstract inspected 26 September 2026.
- National Transportation Safety Board. Assumptions used in the safety assessment process and the effects of multiple alerts and indications on pilot performance. Technical Report ASR-19-01, National Transportation Safety Board, 2019. URL <https://www.nts.gov/investigations/AccidentReports/Reports/ASR1901.pdf>.
- Michael Nygard. Documenting architecture decisions, November 2011. URL <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Full article inspected 26 September 2026.
- OECD. OECD framework for the classification of AI systems. Technical Report 323, OECD Publishing, 2022. URL https://www.oecd.org/en/publications/oecd-framework-for-the-classification-of-ai-systems_cb6d9eca-en.html.
- Erkki Oja. Simplified neuron model as a principal component analyzer. *J. Math. Biol.*, 15(3): 267–273, November 1982. ISSN 1432-1416. doi:[10.1007/BF00275687](https://doi.org/10.1007/BF00275687).
- L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko. *The Mathematical Theory of Optimal Processes*. Wiley Interscience, 1962. URL <https://www.routledge.com/Mathematical-Theory-of-Optimal-Processes/Pontryagin-Boltyanskii-Gamkrelidze-Mishchenko/p/book/9780203749319>. Original English translation; current Routledge reprint page accessed 2026-07-02.

- George R. Price. Selection and Covariance. *Nature*, 227:520–521, August 1970. ISSN 1476-4687. doi:[10.1038/227520a0](https://doi.org/10.1038/227520a0).
- J. A. Robinson. A Machine-Oriented Logic Based on the Resolution Principle. *J. ACM*, 12(1): 23–41, January 1965. ISSN 0004-5411. doi:[10.1145/321250.321253](https://doi.org/10.1145/321250.321253).
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi:[10.1038/323533a0](https://doi.org/10.1038/323533a0).
- Stuart Jonathan Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th, us edition, 2020. ISBN 978-0-13461099-3. URL <https://aima.cs.berkeley.edu/>.
- Secret Society Software, LLC. *CLIPS Reference Manual, Volume I: Basic Programming Guide*, version 6.4.2 edition, 2025. URL <https://clipsrules.net/documentation/v642/bpg642.pdf>.
- Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014. URL <https://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning/understanding-machine-learning-theory-algorithms.pdf>. Author-hosted text, Sections 2.1–2.3 inspected via web extraction, 26 September 2026.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815v1, 2017. URL <https://arxiv.org/abs/1712.01815>. Algorithm description, pp. 2–3, inspected.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. ISBN 9780262039246. URL <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>.
- Elham Tabassi. Artificial intelligence risk management framework (ai rmf 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, January 2023. URL <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>.
- Peter D. Taylor and Leo B. Jonker. Evolutionary stable strategies and game dynamics. *Math. Biosci.*, 40(1):145–156, July 1978. ISSN 0025-5564. doi:[10.1016/0025-5564\(78\)90077-9](https://doi.org/10.1016/0025-5564(78)90077-9).
- Mary Frances Theofanos, Yee-Yin Choong, and Theodore Jensen. AI Use Taxonomy: A Human-Centered Approach. Technical Report NIST AI 200-1, National Institute of Standards and Technology, March 2024. URL <https://www.nist.gov/publications/ai-use-taxonomy-human-centered-approach>.
- Emanuel Todorov and Michael I. Jordan. Optimal feedback control as a theory of motor coordination. *Nat. Neurosci.*, 5:1226–1235, November 2002. ISSN 1546-1726. doi:[10.1038/nn963](https://doi.org/10.1038/nn963).
- U.S.-Canada Power System Outage Task Force. Final report on the august 14, 2003 blackout in the united states and canada. Technical report, U.S.-Canada Power System Outage Task Force, 2004. URL <https://www.energy.gov/sites/prod/files/oeprod/DocumentsandMedia/BlackoutFinal-Web.pdf>.

- U.S. Commodity Futures Trading Commission and U.S. Securities and Exchange Commission. Findings regarding the market events of may 6, 2010. Technical report, CFTC and SEC, 2010. URL <https://www.sec.gov/news/studies/2010/marketevents-report.pdf>.
- U.S. Securities and Exchange Commission. In the matter of knight capital americas llc. Administrative Proceeding File No. 3-15570, Release No. 70694, 2013. URL <https://www.sec.gov/files/litigation/admin/2013/34-70694.pdf>.
- Greg Welch and Gary Bishop. An introduction to the kalman filter. Technical Report TR 95-041, University of North Carolina at Chapel Hill, 2001. URL https://www.cs.cmu.edu/~motionplanning/papers/sbp_papers/kalman/welch_intro_kalman.pdf. Revision dated 8 February 2001; Section 1 inspected.
- Bernard Widrow and Marcian E. Hoff, Jr. Adaptive switching circuits. In *IRE WESCON Convention Record*, volume 4, pages 96–104, 1960. URL <https://isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf>.
- David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi:[10.1109/4235.585893](https://doi.org/10.1109/4235.585893).